

A PRACTICAL GUIDE FOR BUILDERS

Tools & tactics for a *hackathon*.

Choose the right tools, prove the riskiest assumption, ship one complete workflow, and tell a story people remember.

Explore the stack ↓

Exit presentation

6 TOOL CATEGORIES

7 BUILD MOVES

2-MINUTE PITCH

Pick leverage, *not complexity.*

CODE / 01

Codex or Claude Code

Code, debug, refactor, and get oriented in unfamiliar projects faster.

Codex ↗

Claude Code ↗

BUILD / 02

Cursor or Lovable

Move quickly from idea to a convincing interface or full-stack prototype.

Cursor ↗

Lovable ↗

OPEN / 03

Pi or OpenCode

Pair open coding agents with free models through OpenRouter when paid tools are unavailable.

Pi ↗

OpenCode ↗

MODELS / 04

Replicate

Use ready-to-call APIs for image, video, audio, and specialized AI models.

Browse models ↗

DISCOVER / 05

Hugging Face

Find open-source models for vision, speech, text, audio, and generation.

Search models ↗

ROUTE / 06

OpenRouter

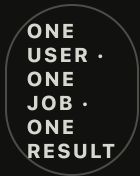
Access multiple free and paid LLM APIs through one consistent integration.

Explore models ↗

Make the core *real first.*

01 Define the smallest realistic core.

Choose
the one
workflow
that
proves
your
idea
from
beginning
to end.



02 Build the riskiest part first.

Test the
AI
model,
external
API,
hardware,
or
uncertain
integration

before
polishing
the
interface.

KILL
UNCERTAINTY
EARLY

03 Ignore non-essential features.

Skip
authentication,
settings,
dashboards,
and
perfect
design
unless
the
demo
truly
depends
on
them.

PROTECT
BUILD
TIME

04 Create one end-to-end flow.

A small
product
that
works
beats a
large
product
with
several
broken
paths.

05 Mock secondary systems.

Use
believable
mock
data
around
the
edges,
while
keeping
the main
innovation
and core
function
real.

MOCK
THE
EDGES,
NOT
THE
IDEA

06 Test continuously.

Keep a
stable,
demoable
version
throughout
the
event
instead
of
integrating
everything
in the

final
hour.

ALWAYS
HAVE A
SAFE
BUILD

07 Add extras only after stability.

Once
the core
is
dependable,
improve
the
experience,
visuals,
and
supporting
features.

POLISH
IS THE
REWARD

Before you build, answer *three questions*.

DOES IT PROVE THE IDEA?

Build it.

If a feature does not prove the core value or unblock the demo, move it to "next."

COULD IT BREAK THE DEMO?

Test first.

Validate the riskiest part before investing time in everything around it.

WHAT MUST BE REAL?

The core.

Make the central innovation work. Supporting data and secondary systems may be mocked.

Tell the story in *two minutes*.

2_{MIN}

20s

Problem + who feels it

15s

Solution in one sentence

50s

Core workflow demo

20s

What is technically different

15s

Impact + what comes next

THE ONE-SENTENCE TEST

“We help [who] solve [problem] by [one clear mechanism].”

Remove every *avoidable risk.*

For the live demo

- ✓ Keep it short and predictable.
- ✓ Avoid typing, logging in, installing, or waiting.
- ✓ Prepare data, windows, and tabs before you present.
- ✓ Keep a recorded backup of the core workflow.

1 min

For the technical video

- ✓ Explain the architecture at a high level.
- ✓ Show the main technologies, models, and data flow.

✓ Skip the tour of every file and library.

THE RULE THAT MATTERS MOST

Simple. *Working.* Clear.



A simple working product with a clear story will usually perform better than an ambitious but unfinished product.

[Back to the start ↑](#)

[Save as PDF](#)

